Q – 1 Tkinter Basic Registration Form

- **Input:-**

```
from tkinter import *

# Main window
root = Tk()
root.title("Tkinter Widgets")
root.geometry("400x400")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)

# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
```

```
# Gender label
gender_lbl = Label(
    root,
    text="Select a Gender:",
    bg="lightblue"
)
gender_lbl.pack(pady=5)

# Variable for radio buttons
gender = StringVar()
gender.set("Male")

r1 = Radiobutton(
    root,
    text="Male",
    variable=gender,
    value="Male",
    bg="lightblue"
)
r1.pack()

r2 = Radiobutton(
    root,
    text="Female",
    variable=gender,
    value="Female",
    bg="lightblue"
)
r2.pack()

# Check button
agree = IntVar()

check = Checkbutton(
    root,
    text="I agree",
    variable=agree,
    font=("Arial", 11),
    bg="lightblue"
```

```
)
check.pack(pady=10)

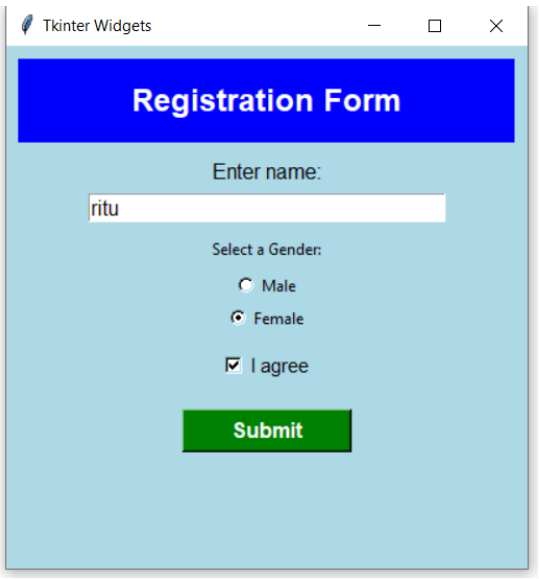
# Button function
def show_data():
    name = name_entry.get()
    print("Name:", name)
    print("Gender:", gender.get())
    print("Agree:", agree.get())

# Submit button
btn = Button(
    root,
    text="Submit",
    command=show_data,
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.pack(pady=10)

# Run the application
root.mainloop()
```

- **Output:-**

```
Name: palak
Gender: Female
Agree: 1
Name: ritu
Gender: Female
Agree: 1
```



Q – 2 Tkinter Registration Form with Country & Password

- **Input:-**

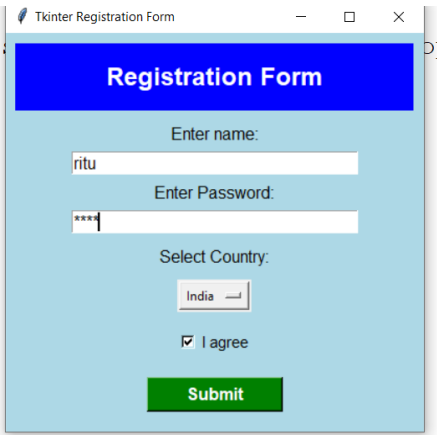
```
from tkinter import *
# Main window
root = Tk()
root.title("Tkinter Registration Form")
root.geometry("400x500")
root.configure(bg="lightblue")
# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)
# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()
# Name entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
# Password label
password_lbl = Label(
    root,
    text="Enter Password:",
```

```
    font=("Arial", 12),
    bg="lightblue"
)
password_lbl.pack()
# Password entry widget
password_entry = Entry(
    root,
    width=30,
    font=("Arial", 12),
    show="*"
)
password_entry.pack(pady=5)
# Country label
country_lbl = Label(
    root,
    text="Select Country:",
    font=("Arial", 12),
    bg="lightblue"
)
country_lbl.pack(pady=5)
# Variable for country
country = StringVar()
country.set("India")
# Country dropdown menu
country_menu = OptionMenu(
    root,
    country,
    "India",
    "USA",
    "UK",
    "Canada",
    "Australia"
)
country_menu.pack(pady=5)
# Check button
agree = IntVar()
check = Checkbutton(
    root,
    text="I agree",
    variable=agree,
```

```
        font=("Arial", 11),
        bg="lightblue"
    )
    check.pack(pady=10)
    # Button function
    def show_data():
        name = name_entry.get()
        password = password_entry.get()
        print("Name:", name)
        print("Password:", password)
        print("Country:", country.get())
        print("Agree:", agree.get())
    # Submit button
    btn = Button(
        root,
        text="Submit",
        command=show_data,
        font=("Arial", 12, "bold"),
        bg="green",
        fg="white",
        width=12
    )
    btn.pack(pady=10)
    # Run the application
    root.mainloop()
```

- **Output:-**

```
== RESTART: C:/U                                    op
Name: palak
Password: 1234
Country: USA
Agree: 1
Name: ritu
Password: 1234
Country: India
Agree: 1
```



Q – 3 Tkinter Registration Form with Language Selection

- **Input:-**

```
from tkinter import *

# Main window
root = Tk()
root.title("Tkinter Registration Form")
root.geometry("400x500")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Registration Form",
    font=("Arial", 18, "bold"),
    fg="white",
    bg="blue",
    width=25,
    height=2
)
lbl.pack(pady=10)

# Name label
name_lbl = Label(
    root,
    text="Enter name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

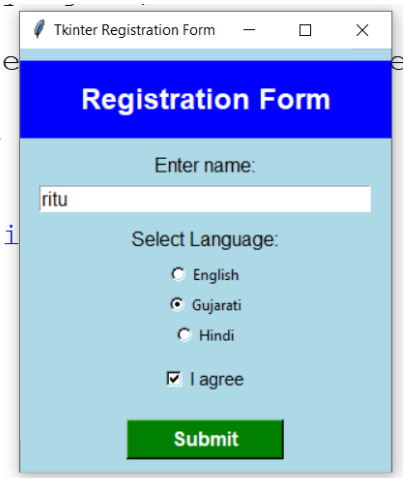
# Entry widget
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)
```

```
# Language label
language_lbl = Label(
    root,
    text="Select Language:",
    font=("Arial", 12),
    bg="lightblue"
)
language_lbl.pack(pady=5)
# Variable for language
language = StringVar()
language.set("English")
# Radio buttons for language selection
r1 = Radiobutton(
    root,
    text="English",
    variable=language,
    value="English",
    bg="lightblue"
)
r1.pack()
r2 = Radiobutton(
    root,
    text="Gujarati",
    variable=language,
    value="Gujarati",
    bg="lightblue"
)
r2.pack()
r3 = Radiobutton(
    root,
    text="Hindi",
    variable=language,
    value="Hindi",
    bg="lightblue"
)
r3.pack()
# Check button
agree = IntVar()
check = Checkbutton(
```

```
    root,
    text="I agree",
    variable=agree,
    font=("Arial", 11),
    bg="lightblue"
)
check.pack(pady=10)
# Button function
def show_data():
    name = name_entry.get()
    print("Name:", name)
    print("Language:", language.get())
    print("Agree:", agree.get())
# Submit button
btn = Button(
    root,
    text="Submit",
    command=show_data,
    font=("Arial", 12, "bold"),
    bg="green",
    fg="white",
    width=12
)
btn.pack(pady=10)
# Run the application
root.mainloop()
```

➢ Output:-

```
>>>
== RESTART: C:/Use
Name: palak
Language: English
Agree: 1
Name: ritu
Language: Gujarati
Agree: 1
```



Q – 4 Student Registration Form with SQLite (CRUD Operations)

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER,
    course TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Student Registration Form")
root.geometry("450x600")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Student Registration Form",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)
```

```
# Student ID label
id_lbl = Label(
    root,
    text="Enter Student ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Student ID entry
id_entry = Entry(root, width=30, font=("Arial", 12))
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(root, width=30, font=("Arial", 12))
name_entry.pack(pady=5)

# Age label
age_lbl = Label(
    root,
    text="Enter Age:",
    font=("Arial", 12),
    bg="lightblue"
)
age_lbl.pack()

# Age entry
age_entry = Entry(root, width=30, font=("Arial", 12))
age_entry.pack(pady=5)
```

```
# Course label
course_lbl = Label(
    root,
    text="Enter Course:",
    font=("Arial", 12),
    bg="lightblue"
)
course_lbl.pack()

# Course entry
course_entry = Entry(root, width=30, font=("Arial", 12))
course_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    age_entry.delete(0, END)
    course_entry.delete(0, END)


# Insert function
def insert_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    course = course_entry.get().strip()

    if not name or not age or not course:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    try:
        age = int(age)
        if age <= 0:
            raise ValueError

        cursor.execute(
            "INSERT INTO students (name, age, course) VALUES (?, ?, ?)",
            (name, age, course)
```

```
            )
            conn.commit()

            messagebox.showinfo("Success", "Student inserted successfully")
            clear_fields()

        except ValueError:
            messagebox.showerror("Error", "Enter a valid age")


    # View function
    def view_data():
        view_win = Toplevel(root)
        view_win.title("Student Records")
        view_win.geometry("500x350")
        view_win.configure(bg="white")

        Label(
            view_win,
            text="Student Records (ID, Name, Age, Course)",
            font=("Arial", 12, "bold"),
            bg="white"
        ).pack(pady=10)

        cursor.execute("SELECT * FROM students")
        records = cursor.fetchall()

        if not records:
            Label(
                view_win,
                text="No student records found",
                bg="white"
            ).pack()
            return

        for record in records:
            Label(
                view_win,
                text=f"ID: {record[0]} | Name: {record[1]} | "
                     f"Age: {record[2]} | Course: {record[3]}",
```

```
            font=("Arial", 10),
            bg="white",
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    student_id = id_entry.get().strip()
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    course = course_entry.get().strip()

    if not student_id or not name or not age or not course:
        messagebox.showwarning(
            "Warning",
            "Enter Student ID, Name, Age and Course"
        )
        return

    try:
        student_id = int(student_id)
        age = int(age)

        if student_id <= 0 or age <= 0:
            raise ValueError

        cursor.execute(
            "UPDATE students SET name=?, age=?, course=? WHERE id=?",
            (name, age, course, student_id)
        )
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success",
                "Student updated successfully"
            )
            clear_fields()
        else:
```

```
            messagebox.showwarning(
                "Warning",
                "Student ID not found"
            )

        except ValueError:
            messagebox.showerror(
                "Error",
                "Student ID and Age must be valid positive numbers"
            )


    # Delete function
    def delete_data():
        student_id = id_entry.get().strip()

        if not student_id:
            messagebox.showwarning(
                "Warning",
                "Enter Student ID to delete"
            )
            return

        try:
            student_id = int(student_id)

            if student_id <= 0:
                raise ValueError

            cursor.execute(
                "SELECT * FROM students WHERE id=?",
                (student_id,)
            )
            record = cursor.fetchone()

            if record is None:
                messagebox.showwarning(
                    "Warning",
                    "Student ID not found"
                )
```

```
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete student {record[1]} (ID: {student_id})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM students WHERE id=?",
                (student_id,)
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Student deleted successfully"
            )
            clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Student ID"
        )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=5)

# View button
Button(
```

```
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=5)

# Update button
Button(
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=5)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=5)

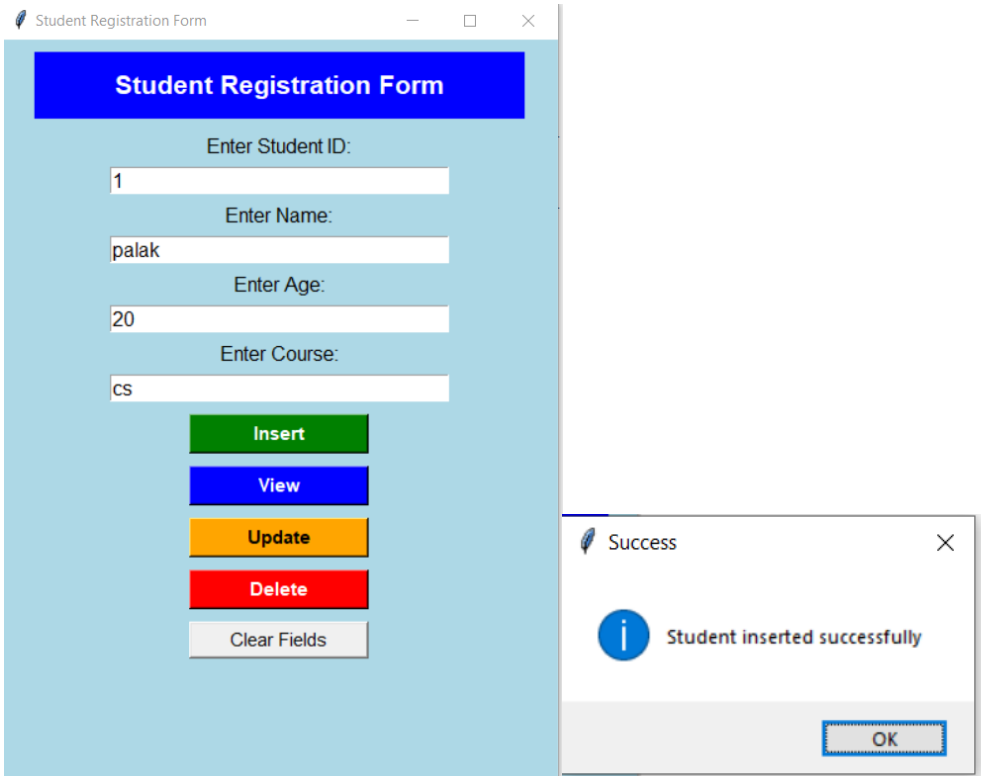
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=5)
```

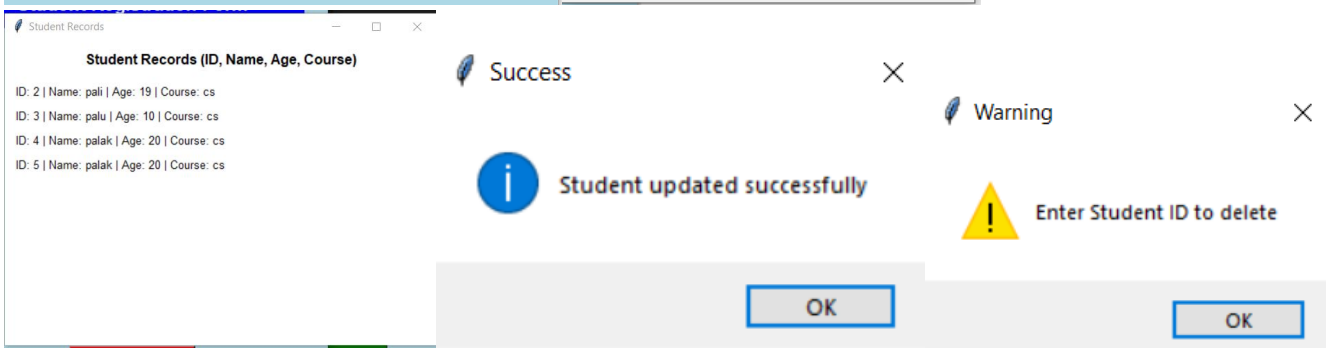
```
# Close database connection
def close_app():
    conn.close()
    root.destroy()


root.protocol("WM_DELETE_WINDOW", close_app)

# Run application
root.mainloop()
```

- **Output:-**





Q – 5 Student Registration with SQLite (College Database)

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("college.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    roll_no INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    semester INTEGER
)
""")
conn.commit()

# Main window
root = Tk()
root.title("College Student Registration")
root.geometry("450x600")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="College Student Registration",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)

# Roll number label
```

```
roll_lbl = Label(
    root,
    text="Enter Roll Number:",
    font=("Arial", 12),
    bg="lightblue"
)
roll_lbl.pack()

# Roll number entry
roll_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
roll_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Student Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)

# Department label
dept_lbl = Label(
    root,
    text="Enter Department:",
    font=("Arial", 12),
    bg="lightblue"
)
```

```
dept_lbl.pack()

# Department entry
dept_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
dept_entry.pack(pady=5)

# Semester label
sem_lbl = Label(
    root,
    text="Enter Semester:",
    font=("Arial", 12),
    bg="lightblue"
)
sem_lbl.pack()

# Semester entry
sem_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
sem_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    roll_entry.delete(0, END)
    name_entry.delete(0, END)
    dept_entry.delete(0, END)
    sem_entry.delete(0, END)


# Insert function
def insert_data():
    roll = roll_entry.get().strip()
    name = name_entry.get().strip()
```

```
    dept = dept_entry.get().strip()
    sem = sem_entry.get().strip()

    if not roll or not name or not dept or not sem:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    try:
        roll = int(roll)
        sem = int(sem)

        if roll <= 0 or sem < 1 or sem > 8:
            raise ValueError

        cursor.execute(
            "INSERT INTO students VALUES (?, ?, ?, ?)",
            (roll, name, dept, sem)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Student registered successfully"
        )
        clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Roll Number and Semester (1-8)"
        )
    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Roll Number already exists"
        )


# View function
def view_data():
```

```
    view_win = Toplevel(root)
    view_win.title("College Student Records")
    view_win.geometry("550x350")

    Label(
        view_win,
        text="College Student Records",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute("SELECT * FROM students ORDER BY roll_no")
    records = cursor.fetchall()

    if not records:
        Label(view_win, text="No student records found").pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"Roll No: {record[0]} | Name: {record[1]} | "
                 f"Department: {record[2]} | Semester: {record[3]}",
            font=("Arial", 10),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    roll = roll_entry.get().strip()
    name = name_entry.get().strip()
    dept = dept_entry.get().strip()
    sem = sem_entry.get().strip()

    if not roll or not name or not dept or not sem:
        messagebox.showwarning(
            "Warning",
            "Enter Roll Number and all new details"
        )
        return
```

```
    try:
        roll = int(roll)
        sem = int(sem)

        if roll <= 0 or sem < 1 or sem > 8:
            raise ValueError

        cursor.execute(
            """UPDATE students
               SET name=?, department=?, semester=?
               WHERE roll_no=?""",
            (name, dept, sem, roll)
        )
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success",
                "Student record updated successfully"
            )
            clear_fields()
        else:
            messagebox.showwarning(
                "Warning",
                "Roll Number not found"
            )

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter valid Roll Number and Semester (1-8)"
        )


# Delete function
def delete_data():
    roll = roll_entry.get().strip()

    if not roll:
```

```
        messagebox.showwarning(
            "Warning",
            "Enter Roll Number to delete"
        )
        return

    try:
        roll = int(roll)

        if roll <= 0:
            raise ValueError

        cursor.execute(
            "SELECT name FROM students WHERE roll_no=?",
            (roll,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
                "Roll Number not found"
            )
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete student {record[0]} (Roll No: {roll})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM students WHERE roll_no=?",
                (roll,)
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Student record deleted successfully"
```

```
            )
            clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Roll Number"
        )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

# View button
Button(
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)

# Update button
Button(
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
```

```
).pack(pady=4)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=4)

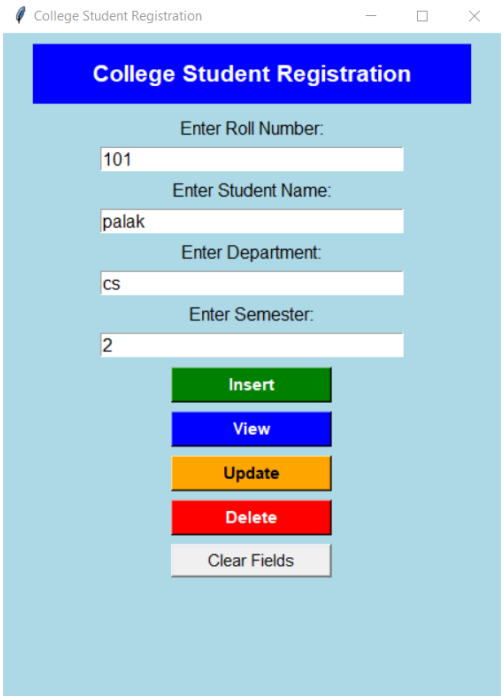
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)

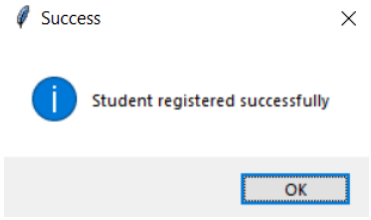

# Close application
def close_app():
    conn.close()
    root.destroy()

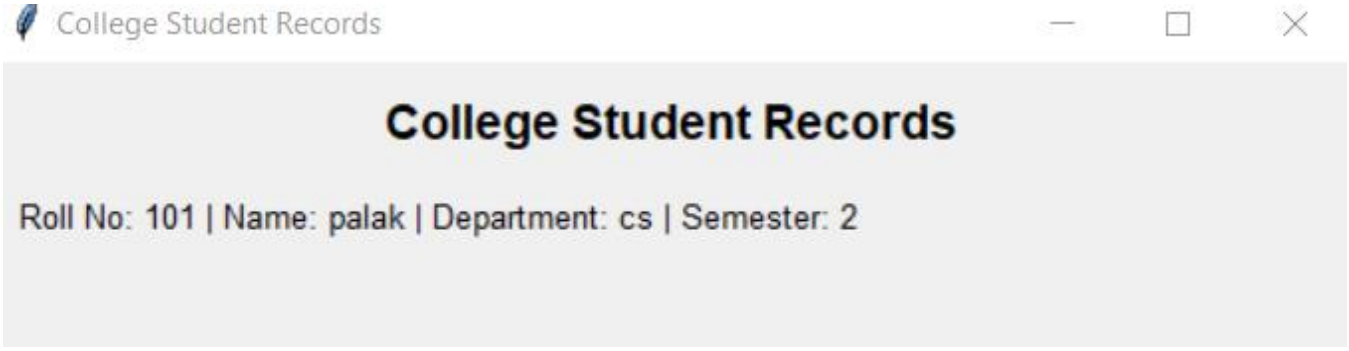

root.protocol("WM_DELETE_WINDOW", close_app)

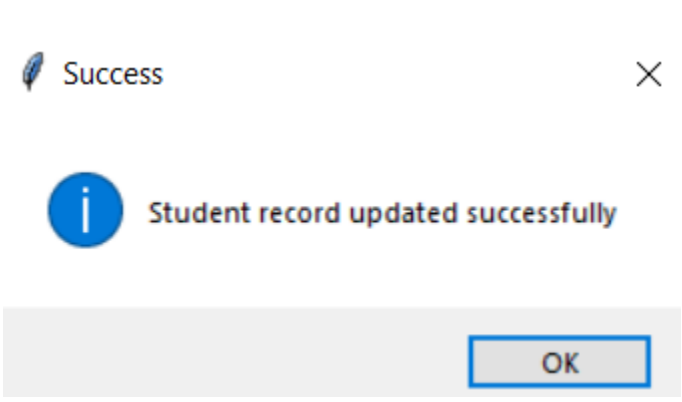
# Run application
root.mainloop()
```

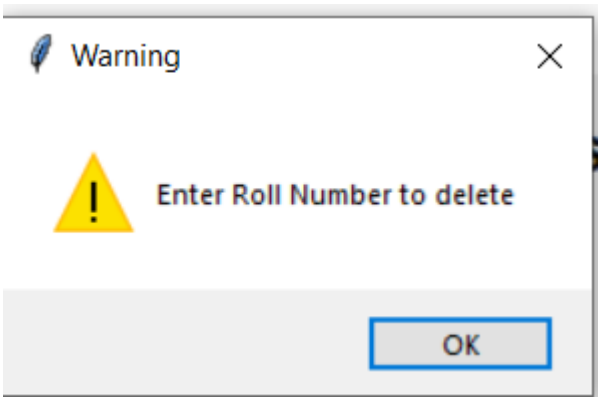
- **Output:-**











Q – 6 Patient Registration System with SQLite

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3

# Database connection
conn = sqlite3.connect("patient.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER,
    gender TEXT,
    disease TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Patient Registration System")
root.geometry("450x650")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Patient Registration System",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)
```

```
# Patient ID label
id_lbl = Label(
    root,
    text="Enter Patient ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Patient ID entry
id_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Patient Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
name_entry.pack(pady=5)

# Age label
age_lbl = Label(
    root,
    text="Enter Age:",
    font=("Arial", 12),
    bg="lightblue"
```

```
)
age_lbl.pack()

# Age entry
age_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
age_entry.pack(pady=5)

# Gender label
gender_lbl = Label(
    root,
    text="Select Gender:",
    font=("Arial", 12),
    bg="lightblue"
)
gender_lbl.pack()

# Gender variable
gender = StringVar()
gender.set("Male")

# Gender radio buttons
r1 = Radiobutton(
    root,
    text="Male",
    variable=gender,
    value="Male",
    bg="lightblue"
)
r1.pack()

r2 = Radiobutton(
    root,
    text="Female",
    variable=gender,
    value="Female",
    bg="lightblue"
```

```
)
r2.pack()

r3 = Radiobutton(
    root,
    text="Other",
    variable=gender,
    value="Other",
    bg="lightblue"
)
r3.pack()

# Disease label
disease_lbl = Label(
    root,
    text="Enter Disease:",
    font=("Arial", 12),
    bg="lightblue"
)
disease_lbl.pack()

# Disease entry
disease_entry = Entry(
    root,
    width=30,
    font=("Arial", 12)
)
disease_entry.pack(pady=5)


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    age_entry.delete(0, END)
    disease_entry.delete(0, END)
    gender.set("Male")


# Insert function
```

```
def insert_data():
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gen = gender.get()
    disease = disease_entry.get().strip()

    if not name or not age or not disease:
        messagebox.showwarning(
            "Warning",
            "Please fill all fields"
        )
        return

    try:
        age = int(age)

        if age < 0 or age > 130:
            raise ValueError

        cursor.execute(
            """INSERT INTO patients (name, age, gender, disease)
               VALUES (?, ?, ?, ?)""",
            (name, age, gen, disease)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Patient registered successfully"
        )
        clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid age between 0 and 130"
        )


# View function
```

```
def view_data():
    view_win = Toplevel(root)
    view_win.title("Patient Records")
    view_win.geometry("600x350")

    Label(
        view_win,
        text="Patient Records",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute("SELECT * FROM patients ORDER BY patient_id")
    records = cursor.fetchall()

    if not records:
        Label(
            view_win,
            text="No patient records found"
        ).pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"ID: {record[0]} | Name: {record[1]} | "
                 f"Age: {record[2]} | Gender: {record[3]} | "
                 f"Disease: {record[4]}",
            font=("Arial", 10),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    patient_id = id_entry.get().strip()
    name = name_entry.get().strip()
    age = age_entry.get().strip()
    gen = gender.get()
    disease = disease_entry.get().strip()
```

```
if not patient_id or not name or not age or not disease:
    messagebox.showwarning(
        "Warning",
        "Enter Patient ID and all patient details"
    )
    return

try:
    patient_id = int(patient_id)
    age = int(age)

    if patient_id <= 0 or age < 0 or age > 130:
        raise ValueError

    cursor.execute(
        """UPDATE patients
           SET name=?, age=?, gender=?, disease=?
           WHERE patient_id=?""",
        (name, age, gen, disease, patient_id)
    )
    conn.commit()

    if cursor.rowcount > 0:
        messagebox.showinfo(
            "Success",
            "Patient record updated successfully"
        )
        clear_fields()
    else:
        messagebox.showwarning(
            "Warning",
            "Patient ID not found"
        )

except ValueError:
    messagebox.showerror(
        "Error",
        "Enter a valid Patient ID and age"
    )
```

```
# Delete function
def delete_data():
    patient_id = id_entry.get().strip()

    if not patient_id:
        messagebox.showwarning(
            "Warning",
            "Enter Patient ID to delete"
        )
        return

    try:
        patient_id = int(patient_id)

        if patient_id <= 0:
            raise ValueError

        cursor.execute(
            "SELECT name FROM patients WHERE patient_id=?",
            (patient_id,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
                "Patient ID not found"
            )
            return

        confirm = messagebox.askyesno(
            "Confirm Delete",
            f"Delete patient {record[0]} (ID: {patient_id})?"
        )

        if confirm:
            cursor.execute(
                "DELETE FROM patients WHERE patient_id=?",
                (patient_id,)
```

```
            )
            conn.commit()

            messagebox.showinfo(
                "Success",
                "Patient record deleted successfully"
            )
            clear_fields()

    except ValueError:
        messagebox.showerror(
            "Error",
            "Enter a valid Patient ID"
        )


# Insert button
Button(
    root,
    text="Insert",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

# View button
Button(
    root,
    text="View",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)

# Update button
Button(
```

```
    root,
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=4)

# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=4)

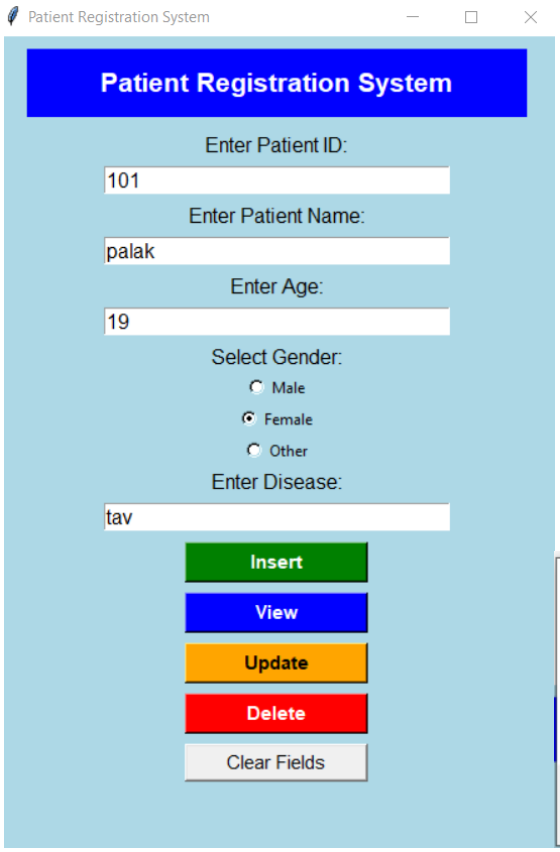
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)

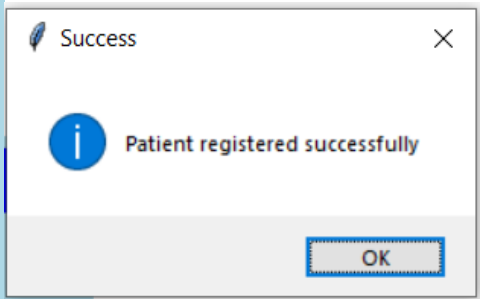

# Close application
def close_app():
    conn.close()
    root.destroy()

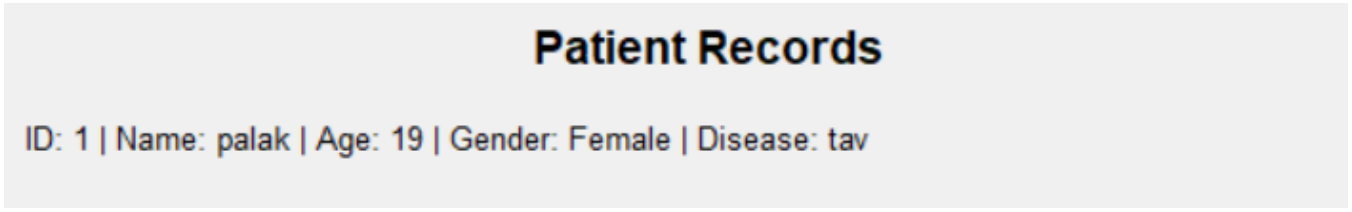
root.protocol("WM_DELETE_WINDOW", close_app)

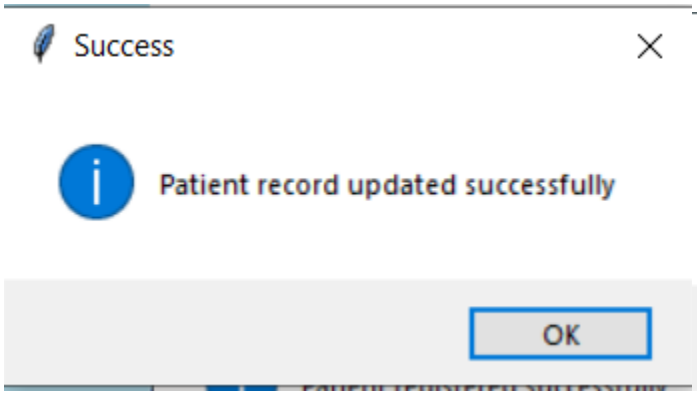
# Run application
root.mainloop()
```

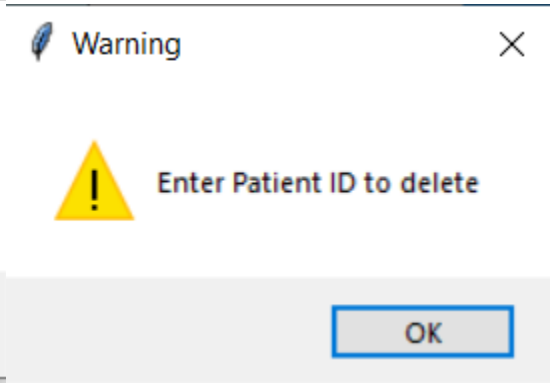
- **Output:-**











Q - 7 Member Registration System with Login Details

- **Input:-**

```
from tkinter import *
from tkinter import messagebox
import sqlite3
import hashlib

# Database connection
conn = sqlite3.connect("member.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT UNIQUE,
    username TEXT UNIQUE,
    password TEXT
)
""")
conn.commit()

# Main window
root = Tk()
root.title("Member Registration System")
root.geometry("450x650")
root.configure(bg="lightblue")

# Heading label
lbl = Label(
    root,
    text="Member Registration System",
    font=("Arial", 16, "bold"),
    fg="white",
    bg="blue",
    width=30,
    height=2
)
lbl.pack(pady=10)

# Member ID label
```

```
id_lbl = Label(
    root,
    text="Enter Member ID:",
    font=("Arial", 12),
    bg="lightblue"
)
id_lbl.pack()

# Member ID entry
id_entry = Entry(root, width=30, font=("Arial", 12))
id_entry.pack(pady=5)

# Name label
name_lbl = Label(
    root,
    text="Enter Name:",
    font=("Arial", 12),
    bg="lightblue"
)
name_lbl.pack()

# Name entry
name_entry = Entry(root, width=30, font=("Arial", 12))
name_entry.pack(pady=5)

# Email label
email_lbl = Label(
    root,
    text="Enter Email:",
    font=("Arial", 12),
    bg="lightblue"
)
email_lbl.pack()

# Email entry
email_entry = Entry(root, width=30, font=("Arial", 12))
email_entry.pack(pady=5)

# Username label
username_lbl = Label(
```

```
    root,
    text="Enter Username:",
    font=("Arial", 12),
    bg="lightblue"
)
username_lbl.pack()

# Username entry
username_entry = Entry(root, width=30, font=("Arial", 12))
username_entry.pack(pady=5)

# Password label
password_lbl = Label(
    root,
    text="Enter Password:",
    font=("Arial", 12),
    bg="lightblue"
)
password_lbl.pack()

# Password entry
password_entry = Entry(
    root,
    width=30,
    font=("Arial", 12),
    show="*"
)
password_entry.pack(pady=5)


# Password hashing function
def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()


# Clear fields function
def clear_fields():
    id_entry.delete(0, END)
    name_entry.delete(0, END)
    email_entry.delete(0, END)
```

```
    username_entry.delete(0, END)
    password_entry.delete(0, END)


# Insert function
def insert_data():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    username = username_entry.get().strip()
    password = password_entry.get()

    if not name or not email or not username or not password:
        messagebox.showwarning("Warning", "Please fill all fields")
        return

    if "@" not in email or "." not in email:
        messagebox.showerror("Error", "Enter a valid email")
        return

    try:
        hashed_password = hash_password(password)

        cursor.execute(
            """INSERT INTO members
              (name, email, username, password)
              VALUES (?, ?, ?, ?)""",
            (name, email, username, hashed_password)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Member registered successfully"
        )
        clear_fields()

    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Email or Username already exists"
```

```
        )


# View function
def view_data():
    view_win = Toplevel(root)
    view_win.title("Member Records")
    view_win.geometry("550x350")

    Label(
        view_win,
        text="Registered Members",
        font=("Arial", 14, "bold")
    ).pack(pady=10)

    cursor.execute(
        "SELECT member_id, name, email, username FROM members"
    )
    records = cursor.fetchall()

    if not records:
        Label(view_win, text="No member records found").pack()
        return

    for record in records:
        Label(
            view_win,
            text=f"ID: {record[0]} | Name: {record[1]} | "
                 f"Email: {record[2]} | Username: {record[3]}",
            font=("Arial", 9),
            anchor="w"
        ).pack(fill="x", padx=10, pady=3)


# Update function
def update_data():
    member_id = id_entry.get().strip()
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    username = username_entry.get().strip()
```

```
password = password_entry.get()

if not member_id or not name or not email or not username:
    messagebox.showwarning(
        "Warning",
        "Enter Member ID, Name, Email and Username"
    )
    return

if "@" not in email or "." not in email:
    messagebox.showerror("Error", "Enter a valid email")
    return

try:
    member_id = int(member_id)

    if password:
        cursor.execute(
            """UPDATE members
               SET name=?, email=?, username=?, password=?
               WHERE member_id=?""",
            (
                name,
                email,
                username,
                hash_password(password),
                member_id
            )
        )
    else:
        cursor.execute(
            """UPDATE members
               SET name=?, email=?, username=?
               WHERE member_id=?""",
            (name, email, username, member_id)
        )

    conn.commit()

    if cursor.rowcount > 0:
```

```
            messagebox.showinfo(
                "Success",
                "Member updated successfully"
            )
            clear_fields()
        else:
            messagebox.showwarning(
                "Warning",
                "Member ID not found"
            )

    except ValueError:
        messagebox.showerror("Error", "Enter a valid Member ID")
    except sqlite3.IntegrityError:
        messagebox.showerror(
            "Error",
            "Email or Username already exists"
        )


# Delete function
def delete_data():
    member_id = id_entry.get().strip()

    if not member_id:
        messagebox.showwarning("Warning", "Enter Member ID")
        return

    try:
        member_id = int(member_id)

        cursor.execute(
            "SELECT name FROM members WHERE member_id=?",
            (member_id,)
        )
        record = cursor.fetchone()

        if record is None:
            messagebox.showwarning(
                "Warning",
```

```
            "Member ID not found"
        )
        return

    confirm = messagebox.askyesno(
        "Confirm Delete",
        f"Delete member {record[0]}?"
    )

    if confirm:
        cursor.execute(
            "DELETE FROM members WHERE member_id=?",
            (member_id,)
        )
        conn.commit()

        messagebox.showinfo(
            "Success",
            "Member deleted successfully"
        )
        clear_fields()

except ValueError:
    messagebox.showerror("Error", "Enter a valid Member ID")


# Login function
def login():
    username = username_entry.get().strip()
    password = password_entry.get()

    if not username or not password:
        messagebox.showwarning(
            "Warning",
            "Enter Username and Password to login"
        )
        return

    cursor.execute(
        "SELECT password FROM members WHERE username=?",
```

```
        (username,)
    )
    record = cursor.fetchone()

    if record and record[0] == hash_password(password):
        messagebox.showinfo(
            "Login Success",
            "Welcome! Login successful"
        )
    else:
        messagebox.showerror(
            "Login Failed",
            "Invalid Username or Password"
        )


# Insert button
Button(
    root,
    text="Register",
    command=insert_data,
    font=("Arial", 11, "bold"),
    bg="green",
    fg="white",
    width=15
).pack(pady=4)

# View button
Button(
    root,
    text="View Members",
    command=view_data,
    font=("Arial", 11, "bold"),
    bg="blue",
    fg="white",
    width=15
).pack(pady=4)
# Update button
Button(
    root,
```

```
    text="Update",
    command=update_data,
    font=("Arial", 11, "bold"),
    bg="orange",
    width=15
).pack(pady=4)
# Delete button
Button(
    root,
    text="Delete",
    command=delete_data,
    font=("Arial", 11, "bold"),
    bg="red",
    fg="white",
    width=15
).pack(pady=)
# Login button
Button(
    root,
    text="Login",
    command=login,
    font=("Arial", 11, "bold"),
    bg="purple",
    fg="white",
    width=15
).pack(pady=4)
# Clear button
Button(
    root,
    text="Clear Fields",
    command=clear_fields,
    font=("Arial", 11),
    width=15
).pack(pady=4)
# Close application
def close_app():
    conn.close()
    root.destroy()
root.protocol("WM_DELETE_WINDOW", close_app)
# Run application
```

```
root.mainloop()
```

- **Output: -**

